# ASSIGNMENT - 2.5

2303A51091
Batch : 29

Task 1: Refactoring Odd/Even Logic (List Version)
❖ Scenario:
You are improving legacy code.
❖ Task:
Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.
❖ Expected Output:
❖ Original and improved code

```python
# Original Version (Legacy Code)

numbers = [10, 15, 20, 25, 30, 35]

even_sum = 0
odd_sum = 0

for i in range(len(numbers)):
    if numbers[i] % 2 == 0:
        even_sum = even_sum + numbers[i]
    else:
        odd_sum = odd_sum + numbers[i]

print("Sum of Even Numbers:", even_sum)
print("Sum of Odd Numbers:", odd_sum)



# Refactored Version (Improved Code)

numbers = [10, 15, 20, 25, 30, 35]

even_sum = sum(num for num in numbers if num % 2 == 0)
odd_sum = sum(num for num in numbers if num % 2 != 0)

print("Sum of Even Numbers:", even_sum)
print("Sum of Odd Numbers:", odd_sum)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py % /usr/bin/python3 "/Users/arepallyra
mcharan/ai assistent.py/assignment-2.5.py/task1.py"
Sum of Even Numbers: 60
Sum of Odd Numbers: 75
Sum of Even Numbers: 60
Sum of Odd Numbers: 75
arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py %
```

Observation:

1. **Readability Improved** – Refactored code is cleaner and easier to understand.
2. **Reduced Code Length** – Fewer lines compared to legacy version.
3. **Better Practice** – Uses Python's built-in `sum()` and direct iteration.
4. **Same Time Complexity** – Both versions run in **O(n)**.
5. **Higher Maintainability** – Less chance of indexing errors and easier to modify.

Task 2: Area Calculation Explanation

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

❖ Expected Output:

➢ Code

➢ Explanation

task2.py > ...

```python
def calculate_area(shape, **dimensions):
    shape = shape.lower()

    if shape == "circle":
        radius = dimensions.get("radius")
        return 3.1416 * radius * radius

    elif shape == "rectangle":
        length = dimensions.get("length")
        width = dimensions.get("width")
        return length * width

    elif shape == "triangle":
        base = dimensions.get("base")
        height = dimensions.get("height")
        return 0.5 * base * height

    else:
        return "Shape not supported"


# Example Usage
print("Circle Area:", calculate_area("circle", radius=5))
print("Rectangle Area:", calculate_area("rectangle", length=4, width=6))
print("Triangle Area:", calculate_area("triangle", base=10, height=8))
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-2.5.py/task2.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py % /usr/bin/python3 "/Users/arepallyra
mcharan/ai assistent.py/assignment-2.5.py/task2.py"
Circle Area: 78.54
Rectangle Area: 24
Triangle Area: 40.0
arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py %
```

Observation:

1. **Modular Design** – One function handles multiple shapes, improving reusability.
2. **Flexible Parameters** – `**dimensions` allows passing different arguments based on shape.
3. **Clear Conditional Logic** – `if-elif` structure makes flow easy to understand.
4. **Scalable Structure** – New shapes can be added easily by adding another condition.
5. **Basic Validation Missing** – No checks for missing or invalid inputs (e.g., negative values or `None`).
6. **Time Complexity** – Constant time **O(1)** since it performs simple calculations.
7.

Task 3: Prompt Sensitivity Experiment

❖ Scenario:

You are testing how AI responds to different prompts.

❖ Task:

Use Cursor AI with different prompts for the same problem and observe

code changes.

❖ Expected Output:

➢ Prompt list

➢ Code variations

task3.py > ...

```python
num = int(input("Enter a number: "))

if num > 1:
    for i in range(2, num):
        if num % i == 0:
            print("Not Prime")
            break
    else:
        print("Prime")
else:
    print("Not Prime")



import math

def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

number = int(input("Enter a number: "))
print("Prime" if is_prime(number) else "Not Prime")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-2.5.py/task3.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-2.5.py/task3.py"
Enter a number: 7
Prime
Enter a number: 9
Not Prime
arepallyramcharan@Arepallys-MacBook-Air assignment-2.5.py %
```

## Observation:

1. Simple prompt → Basic and less optimized code.
2. Detailed prompt → Cleaner and more efficient code.
3. Constraint-based prompt → Better logic and edge-case handling.
4. AI output changes based on prompt clarity and requirements.

Task 4: Tool Comparison Reflection

❖ Scenario:

You must recommend an AI coding tool.

❖ Task:

Based on your work in this topic, compare Gemini, Copilot, and Cursor AI

for usability and code quality.

❖ Expected Output:

Short written reflection

**Gemini**

- Good at explaining code clearly.
- Strong for learning concepts and onboarding beginners.
- Sometimes gives generic or less optimized code unless prompt is detailed.

🔹 **GitHub Copilot**

- Best for real-time code completion inside the IDE.
- Very fast and context-aware.
- Improves productivity but may not always explain logic.

🔹 **Cursor AI**

- Strong at rewriting, refactoring, and modifying entire code blocks.
- Responds well to prompt variations.
- Produces more structured and optimized outputs when given clear instructions.

## Observation:

For beginners → **Gemini** is better for explanations.
For productivity in development → **Copilot** is most efficient.
For prompt-based experimentation and refactoring → **Cursor AI** performs best.

Overall, the best tool depends on the goal: learning, speed, or structured improvements.

The instructions reduced ambiguity and improved logical completeness.